

دانشکده فنی و مهندسی

پایان‌نامه کارشناسی

مهندسی برق (گرایش الکترونیک)

اتوماسیون خانگی با استفاده از بلوتوث و وای فای میکروکنترلر ESP32

دانشجو

ماندانا مرادی - ۹۹۱۵۳۰۲۰۰۷

استاد راهنما

خانم دکتر رفیعی

شهریور ۱۴۰۳

تشکر و قدردانی

منّت خدای را عزّ و جل که طاعتش موجب قربت است و به شکر اندرش مزید نعمت. هر نفسی که فرو می‌رود مُمِدّ حیات است و چون بر می‌آید مُفَرّح ذات. پس در هر نفسی دو نعمت موجود است و بر هر نعمتی شکری واجب.

از دست و زبان که بر آید کز عهده‌ی شکرش به در آید؟

(سعدی)

سپاس خدای را جلّ جلاله که مرا توفیق بخشید تا این مقطع تحصیلی را به اتمام برسانم و آنچه استطاعت دارم به بحر علم، ارزانی دارم. حقگزار زحمات استاد گرانقدر خویش، خانم دکتر رفیعی هستم که مرا در مسیر انجام این پروژه یاری دادند.

ماندانا مرادی

۲۷ شهریور ۱۴۰۳

چکیده

در این پروژه، در فصل اول به معرفی سخت افزار و نحوه‌ی اتصال اجزاء و ماژول‌ها به میکروکنترلر پرداخته شده است. در فصل دوم به نحوه‌ی نوشتن کد پروژه پرداخته شده است. در فصل سوم نتایج نهایی پروژه ارائه شده است. در فصل چهارم پیشنهاداتی برای انجام پروژه‌های مشابه و یا پیشرفت دادن همین پروژه داده شده است. در فصل پنجم نحوه‌ی نصب و راه اندازی نرم افزارهای مورد نیاز جهت برنامه نویسی و ارسال اطلاعات از طریق بلوتوث و صفحه‌ی وب، برای برد و دریافت خروجی‌های پروژه، توضیح داده شده.

کلیدواژه‌ها: بلوتوث، وای فای، اتوماسیون خانگی، Home Automation، ESP32

فهرست مطالب

عنوان صفحه

فصل ۱

سخت افزار ۱

۱.۱ تاریخچه‌ی برد ESP32 ۱

۱.۱.۱ بلوتوث میکروکنترلر ESP32 ۲

۲.۱ اجزای پروژه ۳

۱.۲.۱ میکروکنترلر ESP32 ۴

۲.۲.۱ ماژول رله ۴ کاناله ۵

۳.۲.۱ سروو موتور TowerPro SG90 360 ۶

۴.۲.۱ نمایشگر LCD 20x4 ۷

۵.۲.۱ راه انداز Interface I2C 20x4 Character LCD ۷

۳.۱ اتصالات مدار ۸

فصل ۲

نحوه‌ی نوشتن برنامه‌ی این پروژه ۹

۱.۲ بخش وای فای ۱۰

۱.۱.۲ ابتدای برنامه خارج از void setup() و void loop () ۱۲

۲.۱.۲ در تابع void setup() ۱۲

۳.۱.۲ در تابع void loop() ۱۲

۲.۲ بخش بلوتوث ۱۰

۱.۲.۲ ابتدای برنامه خارج از void setup() و void loop () ۱۶

۲.۲.۲ در تابع void setup() ۱۷

۳.۲.۲ در تابع void loop() ۱۷

۲.۲ توضیح LCD ۲۰

۲.۲ توضیح سروو موتور ۱۰

فصل ۳

نتایج نهایی پروژه ۲۰

۱.۳ نتایج نهایی پروژه ۲۰

ادامه‌ی فهرست مطالب

عنوان صفحه

فصل ۴

پیشنهادات..... ۲۴

۱.۴ پشنهادات..... ۲۴

فصل ۵

پیش نیازهای نرم افزاری..... ۲۳

۱.۵ نصب Arduino IDE..... ۲۳

۲.۵ نصب درایور نرم افزار CP2102..... ۲۹

۳.۵ نصب نرم افزار موبایل برای ارتباط با بلوتوث..... ۳۰

مراجع..... ۳۲

فهرست شکل‌ها

عنوان **صفحه**

شکل (۱-۱) میکرو کنترلر .ESP32 ..... ۴
شکل (۱-۲) پایه‌های میکروکنترلر ESP32 ..... ۵
شکل (۱-۳) ماژول رله‌ی ۴ کاناله ۵
شکل (۱-۴) سروو موتور ۶
شکل (۱-۵) Character LCD ۷
شکل (۱-۶) **ارتباط** I2C **با** Character LCD ۷
شکل (۱-۷) **اتصالات** Character LCD ۹
شکل (۱-۸) شماتیک مدار ۹
شکل (۲-۱) پایه‌های SDA و SCL میکروکنترلر ۷
شکل (۵-۱) نصب کتابخانه **ESP32** ۲۳
شکل (۵-۲) انتخاب برد uPesy WROOM DevKit ۲۴
شکل (۵-۳) افزایش اندازه حافظه اختصاص داده شده به کد ۲۴
شکل (۵-۴) نصب کتابخانه‌ی **ESP32Servo.h** ۲۵
شکل (۵-۵) نصب کتابخانه‌ی **LiquidCrystal_I2C** ۲۵
شکل (۵-۶) تغییر فرکانس **flash frequency** ۲۶
شکل (۵-۷) تغییر فرکانس **CPU** به **MHz** ۱۶۰ ..... ۲۶
شکل (۵-۸) تنظیم بادریت پروگرمینگ ۲۷
شکل (۵-۹) ایجاد فایل باینری کد ۲۷
شکل (۵-۱۰) فایل باینری قابل فلش روی برد ۲۸
شکل (۵-۱۱) نحوه‌ی باز کردن **serial monito** در **arduino IDE** ۲۸
شکل (۵-۱۲) مشاهده شماره **COM** در **Device Manager** ۲۹
شکل (۵-۱۳) انتخاب شماره **COM** در **Arduino IDE** ۳۰
شکل (۵-۱۴) آیکون نرم افزار **Serial Bluetooth Terminal** ۳۰
شکل (۵-۱۵) نرم افزار **serial bluetooth terminal** ۳۱

فهرست جداول

عنوان **صفحه**

جدول (۴-۱) برآورد هزینه پروژه..... ۲۲

فصل ۱

سخت افزار:

در این پروژه از میکروکنترلر ESP32، ماژول رله‌ی ۴ کاناله، آداپتور ۵ ولت ۲ آمپر، ۴ لامپ ۱۵ وات و دوشاخه استفاده شده است.

۱.۱ تاریخچه‌ی برد ESP32:

شرکت esspressif systems در سال ۲۰۰۸ در چین تاسیس شد. این شرکت از یک استارتاپ کوچک رشد کرد و در صنعت نیمه هادی، به یک شرکت پیشتاز، در بحث اینترنت اشیا، در جهان تبدیل شد.

در سال ۲۰۱۴ Espressif Systems تراشه‌ی ESP8266 را تولید کرد. ESP8266 یک میکروکنترلر Wi-Fi ارزان قیمت است که در آن نرم افزار شبکه TCP/IP تعبیه شده است. این تراشه در آگوست ۲۰۱۴ از طریق ماژول ESP-01 که توسط سازنده شخص ثالث Ai-Thinker ساخته شده بود، در جامعه سازندگان انگلیسی زبان رایج شد. این ماژول کوچک به میکروکنترلرها اجازه می دهد تا به یک شبکه Wi-Fi متصل شوند و با استفاده از دستورات سبک Hayes اتصالات ساده TCP/IP را ایجاد کنند.

مجموعه دستورات هایس (همچنین به عنوان مجموعه دستورات AT Command شناخته می شود) یک زبان دستوری خاص است که در ابتدا توسط دیل هدرینگتون و دنیس هیز برای مودم هوشمند Hayes Smartmodem 300 baud در سال ۱۹۸۱ توسعه یافت.

پس از موفقیت چشمگیرESP8266 ، شرکت Espressif در سال ۲۰۱۶، در یک ارتقاء کامل ESP32 را منتشر کرد. این میکروکنترلر نه تنها WiFi بلکه بلوتوث ۴.۰ (BLE/Bluetooth Smart) را نیز در خود جای داده است که آن را برای هر برنامه اینترنت اشیا (IoT) ایده آل می کند.

برد های ESP32 دارای امکانات بسیار خارق العاده ای هستند. این میکروکنترلر از نسل قبلی خود یعنی ESP8266 بسیار بهتر عمل کرده و در حقیقت نسخه پیشرفته تر آن است. یکی از مزایای ESP32 این است که GPIO های بسیار بیشتری نسبت به ESP8266 دارد. و شما مجبور نخواهید بود که پین های IO خود را مالتیپلکس کنید.

ESP32 مجموعه‌ی کاملی از تجهیزات جانبی (peripheral) را دربرمی‌گیرد، از سنسورهای لمسی خازنی، رابط کارت SD، اترنت، SPI پرسرعت، UART، I2S و I2C.

۱.۱.۱ بلوتوث میکروکنترلر ESP32:

ماژول بردهای ESP32 از بلوتوث دو حالته پشتیبانی می‌کنند، به این معنا که هم از بلوتوث کلاسیک (Bluetooth Classic) و هم از بلوتوث کم مصرف BLE (Bluetooth Low Energy) پشتیبانی می‌کنند. با اینکه این دو پروتکل در موارد بسیار مهمی با یکدیگر اشتراک دارند؛ از جمله معماری و اینکه هر دو در باند فرکانسی ۲.۴ گیگاهرتز (ISM) فعالیت می‌کنند، اما لازم است بدانید که این دو کاملاً با یکدیگر متفاوت‌اند.

تکنولوژی اصلی بلوتوث، که ما از مدت ها قبل با آن از طریق اتصال تلفن‌های هوشمند خود به هدفون‌های بی‌سیم آشنا هستیم، به عنوان بلوتوث کلاسیک شناخته می‌شود. اگر تا به حال ماژول‌های بلوتوث به سریال مانند HC-05یا HC-06 را با آردوینو مورد استفاده قرار داده‌اید، ناخودآگاه از بلوتوث کلاسیک استفاده کردید!

بلوتوث کلاسیک برای انتقال داده دوطرفه به صورت پیوسته و برای کاربردهای با گذردهی بالا (حداکثر ۳ مگابیت بر ثانیه) طراحی شده‌است، اما تنها برای مسافت‌های کوتاه مناسب می‌باشد. همچنین از بیش از ۷۹ کانال در باند فرکانسی 2.4گیگاهرتز ISM استفاده می‌کند. این نوع بلوتوث عمدتاً در پروژه‌هایی به کار می‌رود که شما نیاز به انتقال مداوم داده دارید، مانند انتقال فایل یا صوتی.

بلوتوث کم مصرف (BLE) چیست؟

بلوتوث کم مصرف که ابتدا به عنوان بلوتوث هوشمند به بازار عرضه شد و به طور رایج به آن BLE گفته می‌شد، در واقع با حفظ محدوده ارتباطی مشابه نوع کلاسیک، برای کاربردهایی با توان مصرفی بسیار کم طراحی شده است. با این وجود، BLEبیش از یک نسخه‌ی کم مصرف از بلوتوث کلاسیک است BLE .در طیف رادیویی مشابهی مانند بلوتوث کلاسیک فعالیت می‌کند، اما از ۴۰ کانال مختلف با نرخ انتقال داده‌ی کمتر (حداکثر ۱ مگابیت بر ثانیه) استفاده می‌کند.

این نوع بلوتوث برای پروژه‌های اینترنت اشیا که توان مصرفی یکی از نگرانی‌های اصلی آنهاست، بسیار مناسب می‌باشد. برای مثال، پروژه‌ای که باید در آن دستگاه‌ها را به صورت دوره‌ای بیدار کنید، داده‌های سنسور را جمع‌آوری و آن‌ها را از طریق بلوتوث انتقال دهید و سپس دستگاه‌ها را به حالت خواب بازگردانید.

مقایسه بلوتوث کم مصرف BLE و بلوتوث کلاسیک

ارتباط بلوتوث نوع BLE در درجه اول برای ذخیره توان مصرفی مورد استفاده قرار می‌گیرد، اما چندین تفاوت مهم وجود دارد که در زیر بیان شده است.

- **توان مصرفی**

معمولاً بلوتوث کلاسیک توان مصرفی بیشتری (حدود ۱ وات) دارد، درحالی که BLEبا توان مصرفی کم (بین ۰.۰۱ تا ۰.۵ وات) طراحی شده‌است و آن را برای استفاده در دستگاه‌هایی که تغذیه آن‌ها از طریق باتری تأمین می‌شود و نیاز به فعالیت در دوره‌های طولانی را دارند، بسیار مناسب ساخته‌است.

- **نرخ انتقال داده**

بلوتوث کلاسیک نرخ انتقال داده‌ی بیشتری را نسبت به BLE فراهم می‌کند که آن را برای پروژه‌هایی که نیازمند انتقال داده پیوسته و با سرعت بالا هستند مناسب ساخته‌است، درحالی که BLE برای انتقال مجموعه داده‌های کوتاه مناسب است.

- **مسافت**

هر دو بلوتوث کلاسیک و BLE می‌توانند تا حداکثر ۱۰۰ متر را پوشش دهند، اما فاصله‌ی دقیق با توجه به محیط و پیاده‌سازی‌ها تغییر می‌کند.

- **تأخیر**

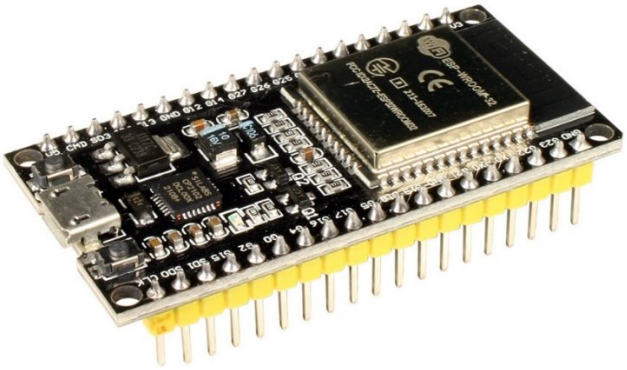
ارتباط مبتنی بر بلوتوث کلاسیک حداکثر ۱۰۰ میلی ثانیه تأخیر خواهد داشت، در حالی که ارتباط مبتنی بر BLEتأخیری حدود ۶ میلی ثانیه خواهد داشت .توجه داشته باشید هر چه تأخیر کمتر باشد، بهتر است.

- **سازگاری**

بلوتوث کلاسیک بیشتر در دستگاه‌های قدیمی استفاده شده‌است، در حالی که BLEدر تلفن‌های همراه هوشمند و سایر دستگاه‌های ارتباطی مدرن به کار رفته‌است.

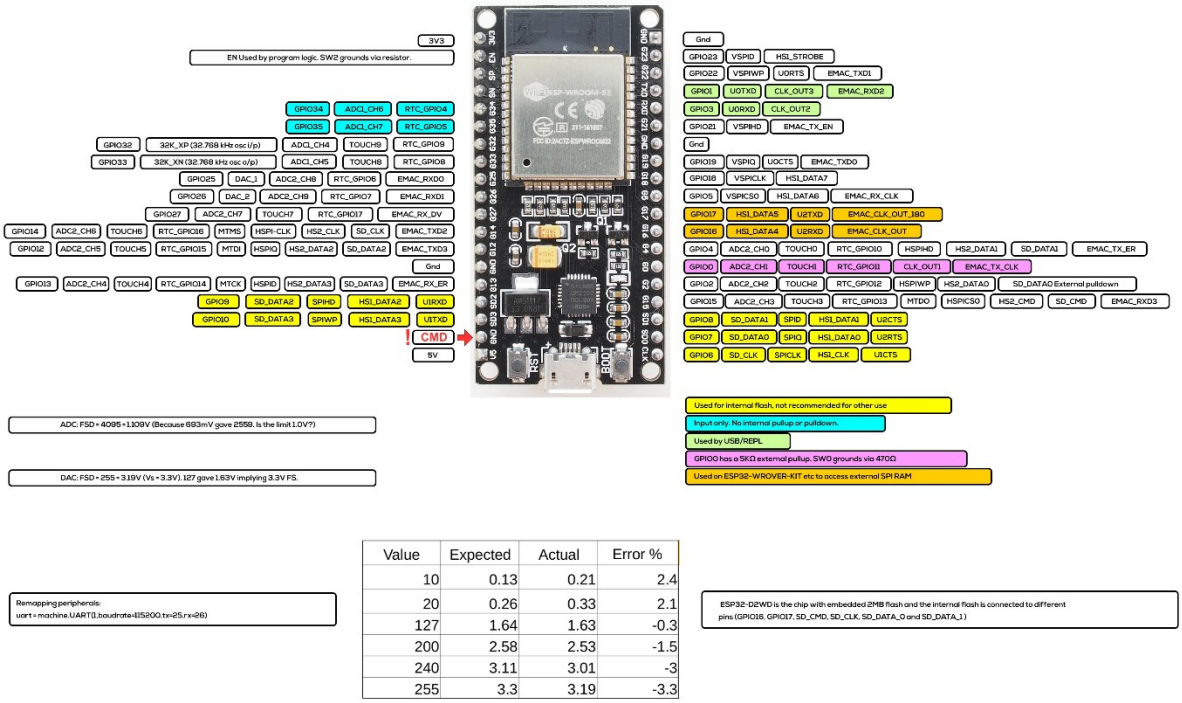
۲.۱ اجزای پروژه:

۱.۲.۱ میکرو کنترلر ESP32:



شکل (۱-۱) میکروکنترلر ESP32

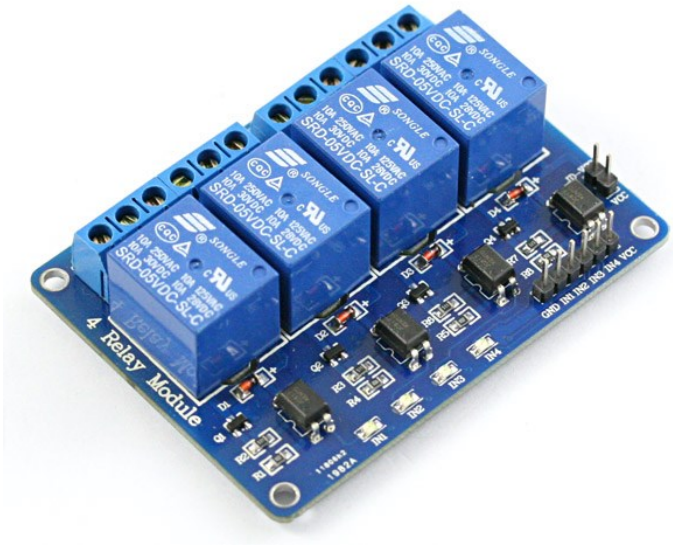
این میکروکنترلر با ولتاژ ۳ ولت کار می کند اما درون آن یک رگولاتور وجود دارد و می توان به پایه ای که کنار آن 5V نوشته شده، برای تغذیه میکروکنترلر ولتاژ ۵ ولت اعمال کرد. به دیگر پایه های میکرو کنترلر هرگز نباید ولتاژی بالاتر از ۳ ولت اعمال شود. روی این میکروکنترلر یک CP2102 برای ارتباط سریال با کامپیوتر قرار داده شده. بدین منظور باید از کابل Micro-USB استفاده شود. دقت شود از کابل‌های شارژی که قابلیت انتقال دیتا ندارند نمی توان استفاده کرد. در صورت وصل بودن کابل، تغذیه برد از ۵ ولت USB کامپیوتر تامین می شود و نباید به پایه ۵ ولت یا ۳ ولت آن ولتاژی جهت تغذیه کردن برد اعمال شود. چرا که می تواند موجب آسیب دیدن برد یا منبع تغذیه و یا هردو گردد. برای بردن میکروکنترلر به Programming Mode ابتدا دکمه‌ی Boot را فشرده و سپس دکمه‌ی Reset فشرده شود.



Value	Expected	Actual	Error %
10	0.13	0.21	2.4
20	0.26	0.33	2.1
127	1.64	1.63	-0.3
200	2.58	2.53	-1.5
240	3.11	3.01	-3
255	3.3	3.19	-3.3

شکل (۱-۲) پایه‌های میکروکنترلر ESP32

۲.۲.۱ ماژول رله ۴ کاناله:



شکل (۱-۳) ماژول رله‌ی ۴ کاناله

تغذیه این رله با ولتاژ ۵ ولت انجام می شود. هر رله ۳ سر دارد. از این ۳ یکی مشترک است، یکی Normally open، دیگری Noramally Closed است. پایه سیگنال رله معمولا Intenally Pull-Up است و برای فعال شدن رله باید به این پایه صفر داده شود(یعنی Active Low است). ولتاژ پایه های سیگنال رله پیش از آنکه چیزی به آن داده شود برابر ۳ ولت است؛ بنابراین با میکروکنترلر هایی که با ولتاژ ۳ ولت کار می کنند (همچون ESP32) و همچنین با میکرو کنترلر هایی که با ۵ ولت کار می کنند (همچون STM32، ATmega32) قابل استفاده است.

۳.۲.۱ سروو موتور TowerPro SG90 360:



شکل (۱-۴) سروو موتور

سروو موتور TowerPro SG90 360 (شکل ۱-۵)، چرخش ۳۶۰ درجه دارد، سیم قرمزرنگ آن به ۵V و سیم قهوه‌ای‌رنگ آن به زمین و سیم نارنجی‌رنگ آن به PWM میکروکنترلر متصل می شود و با تغییر Duty Cycle پالس PWM، سرعت موتور تغییر می کند.

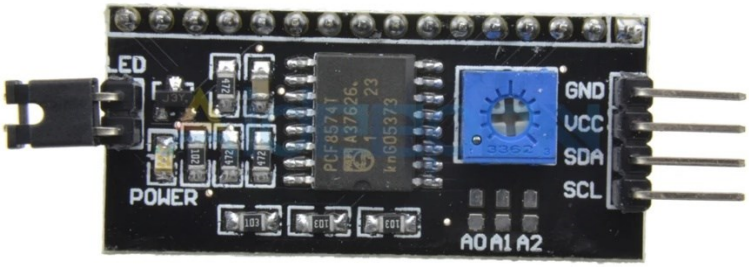
برای تغذیه این موتور می توان از ولتاژ ۴.۸ تا ۶ ولت استفاده کرد. پالس PWM آن می‌تواند ۳.۳ و یا ۵ ولت باشد اما فرکانس این پالس باید همواره ۵۰ هرتز باشد.

۴.۲.۱ نمایشگر LCD 20x4:



شکل (۱-۵) Character LCD

۵.۱.۱ راه انداز Character LCD 16x2 I2C Interface:



شکل (۱-۶) ارتباط I2C با Character LCD

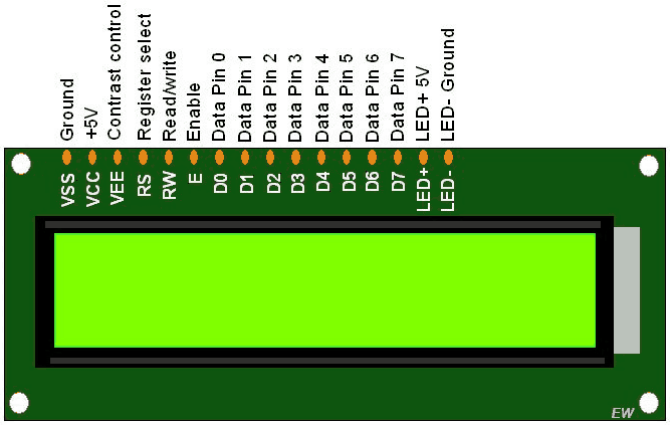
LCDهای کاراکتری 20x4 ، ۱۶ پایه دارند، ارسال اطلاعات به آن‌ها می توانند به سه حالت ۸ بیتی، ۴ بیتی و به صورت سریال (از طریق I2C) انجام شود. ارسال اطلاعات به صورت سریال کمک می کند پایه‌های کمتری از میکروکنترلر مصرف شود.

پایه های یک پتانسیومتر ۱۰K به VSS، VCC و VEE متصل شوند. به پایه‌های کناری پتانسیومتر ۵V و زمین متصل شوند. همچنین به VSS زمین و به ۵V VCC متصل شود (شکل ۱-۸). پایه وسط پتانسیومتر به VEE متصل شود. با چرخاندن این پتانسیومتر contrast نمایشگر تنظیم می شود و وضعیت بهینه برای دیدن کاراکترها، قابل تنظیم می‌شود.

K به زمین و A به ۵V متصل شود به این صورت Backlight نمایشگر تامین می شود.

EN و RS برای تعیین مود ها استفاده می شوند و باید به میکروکنترلر متصل شوند. در صورتی که چیزی از LCD نمی‌خوانیم RW را می‌توان زمین کرد.

در مود ۸ بیتی از D0تا D8 و در مورد ۴ بیتی از D4 تا D8 برای ارسال اطلاعات استفاده می‌شود.

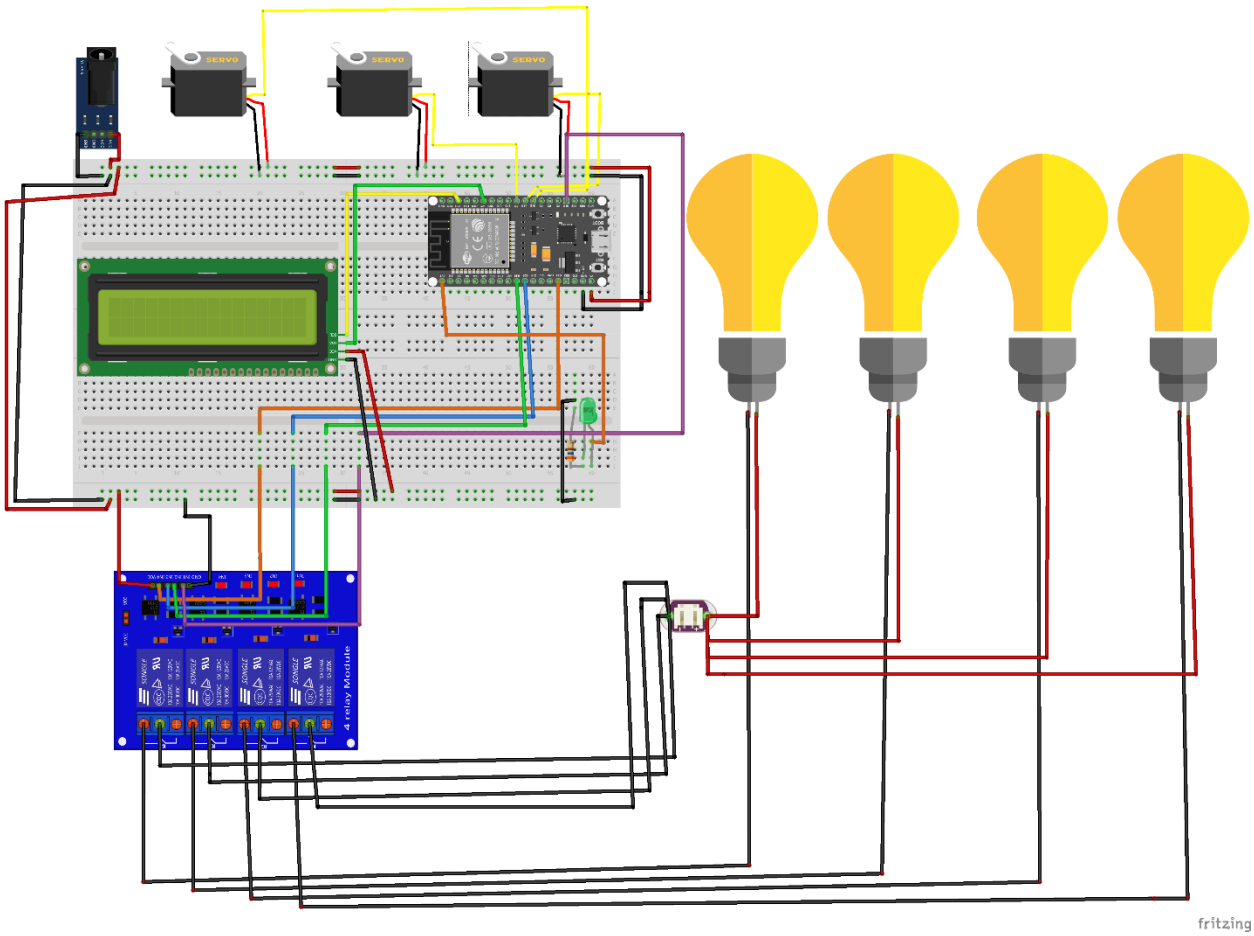


شکل (۱-۶) اتصالات Character LCD

در صورت اتصال ماژول I2C (شکل ۱-۷) احتیاجی به اتصالات دو خط بالاتر نیست چرا که در مدار داخلی ماژول انجام شده است. در ماژول I2C فقط VCC به ۵ و GND به زمین متصل شود و طبق دیتا شیت سازنده کامندها و کاراکترهای مورد نظر، به صورت سریال از طریق I2C برای ماژول ارسال شود.

۳.۱ اتصالات مدار:

اتصالات مدار به شکل زیر انجام شد.



شکل (۱-۸) شماتیک مدار

فصل دوم:

نوشتن برنامه این پروژه

در Arduino IDE دو تابع وجود دارد به نام‌های void setup() و void loop() وجود دارند. کدهای تابع اول یکبار اجرا می‌شوند (مثل خروجی کردن پایه‌ها) و کدهای تابع دوم بی نهایت بار تکرار می شوند. در ابتدای کد و پیش از این دو تابع می توان هدر فایل‌ها را اضافه کرد و یا برخی تعارف را انجام داد.

۱.۲ بخش وای فای:

ماژول‌ها و میکروکنترلرهای مجهز به وای فای معمولا سه حالت کاری access point، station و یا هر دو به صورت همزمان را دارا هستند. Access point یعنی ماژول برای دیگر دستگاه‌های مجهز به وای فای قابل دیدن، دسترسی و اتصال است. station به این معناست که ماژول به یک مودم (router) متصل می‌شود و از آن IP می‌گیرد. بقیه دستگاه ها هم باید به همین مودم متصل شوند و با وارد کردن IP ماژول در Browser خود (مانند firefox و یا chrome) می توانند صفحه وب طراحی شده در این ماژول را (که به زبان HTML و CSS انجام می‌شود) ببینند.

در این پروژه از حالت station استفاده شده است.

۱.۱.۲ ابتدای برنامه، خارج از void setup() و void loop():

هدر فایل‌های زیر افزوده شوند.

```
#include <WiFi.h> // کتابخانه برای اتصال به شبکه WiFi
#include <WiFiClient.h> // کتابخانه برای مدیریت اتصالات WiFi
#include <WebServer.h> // کتابخانه برای ایجاد یک وب سرور
#include <uri/UriBraces.h> // کتابخانه برای پردازش آدرس‌های URL با قواعد خاص
```

اعداد زیر، مربوط به GPIOهای متصل به رله‌ها تعریف شوند.

```
const int LED1 = 2; // شماره پین برای LED ۱
const int LED2 = 26; // شماره پین برای LED ۲
const int LED3 = 27; // شماره پین برای LED 3
const int LED4 = 13; // شماره پین برای LED 4
```

چهار متغیر زیر به صورت Boolean تعریف شوند. بعدا با فشرده شدن دکمه‌ها در صفحه وب مقدار این متغیرها صفر و یک می‌شود و حالت رله را تغییر می‌دهد و لامپ ها روشن و خاموش خواهند شد.

```
bool led1State = true; // وضعیت فعلی LED ۱
bool led2State = true; // وضعیت فعلی LED ۲
bool led3State = true; // وضعیت فعلی LED 3
bool led4State = true; // وضعیت فعلی LED 4
bool led5State = true; // وضعیت فعلی LED 5
```

تعریفات زیر انجام شد. WIFI_SSID به معنای نام یک دستگاه مجهز به وای فای است. در این برنامه باید نام یک مودم (router) و پسورد آن داده شود تا میکروکنترلر ESP32 به آن مودم متصل شود و از آن IP در یافت کند.

```
#define WIFI_SSID "mahan" // نام شبکه WiFi
```

```
#define WIFI_PASSWORD "Gnce_nt55" // رمز عبور شبکه WiFi
// برای افزایش سرعت اتصال WiFi تعریف کانال:
#define WIFI_CHANNEL 6
WebServer server(80); // ایجاد یک شیء وب سرور با پورت ۸۰
```

یک تابع به نام `void sendHTML()` تعریف شد. در این تابع ویژگی‌های صفحه‌ی وب تعیین خواهند شد.

```
void sendHtml() {
  String response = R"(
    <!DOCTYPE html><html>
      <head>
        <title> Ac Power Controller </title>
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <style>
          html { font-family: sans-serif; text-align: center; }
          body { display: inline-flex; flex-direction: column; }
          h1 { margin-bottom: 1em; }
          h1 { margin-top: 1em; }
          h1 { margin-right: 1em; }
          h1 { margin-left: 1em; }
          h3 { margin-top: 1em;}
          h3 { margin-bottom: 1em;}
          h3 { margin-right: 0em;}
          h3 { margin-left: 0em;}
          h3 {font-size: 1.75em; }
.aa{
width: 100% !important;
}
   #div1, #div2 , #div3  , #div4 ,#div5{
             width: 200px;
             height: 250px;
             float: left;
        }
             *.btn {
                 background-color: #5B5;
                   width: 200px;
                    border: none; color: #fff; padding: 1em ;
                  font-size: 1.75em; text-decoration: none;
             }
            *.btn.OFF { background-color: #333; }
        </style>
      </head>
```

```
      <body>
          <div class="aa"> <h1> 4
          Ac Device Power Controller Developed By Mandana Moradi
          </h1> </div>
          <div class="aa" >
            <div id="div1"> <h3>Device 1</h3>
          <a href="/toggle/1" class="btn LED1_TEXT">LED1_TEXT</a></div>
            <div id="div2"><h3>Device 2</h3>
          <a href="/toggle/2" class="btn LED2_TEXT">LED2_TEXT</a></div>
            <div id="div3"> <h3>Device 3</h3>
          <a href="/toggle/3" class="btn LED3_TEXT">LED3_TEXT</a></div>
            <div id="div4"><h3>Device 4</h3>
          <a href="/toggle/4" class="btn LED4_TEXT">LED4_TEXT</a></div>
            <div id="div5"><h3>PWM</h3>
          <a href="/toggle/5" class="btn LED5_TEXT">LED5_TEXT</a></div>

         </div>
      </body>
    </html>
  )"; // ایجاد صفحه HTML با دکمه‌های کنترل برای LEDها
   // جایگزینی متن دکمه‌ها با وضعیت فعلی هر LED
  response.replace("LED1_TEXT", led1State ? "OFF" : "ON");
  response.replace("LED2_TEXT", led2State ? "OFF" : "ON");
  response.replace("LED3_TEXT", led3State ? "OFF" : "ON");
  response.replace("LED4_TEXT", led4State ? "OFF" : "ON");
  response.replace("LED5_TEXT", led5State ? "OFF" : "ON");
  // ارسال صفحه HTML به مشتری
  server.send(200, "text/html", response);
}
```

یک string با نام String response = R تعریف شد.

۲.۱.۲ در تابع void setup():

برای ارتباط سریال با برد، بادریت ۱۱۵۲۰۰ تعیین شد. یعنی ESP32 اطلاعاتی را به صورت سریال (با استفاده از cp2102 که روی برد تعبیه شده) برای کامپیوتر ارسال می‌کند و با باز کردن serial monitor در Arduino IDE می‌توان اطلاعات را مشاهده کرد.

```
Serial.begin(115200); // شروع ارتباط با سریال با سرعت ۱۱۵۲۰۰ بیت بر ثانیه
```

پایه‌های متصل به رله‌ها به صورت خروجی تعریف شد و با توجه به اینکه رله‌ها صفر فعال هستند و مطلوب است حالت اولیه آن ها غیر فعال (خاموش) باشد مقدار اولیه این پایه‌ها را روشن (HIGH) در نظر گرفته شد.

```
pinMode(LED1, OUTPUT); // تنظیم پین LED ۱ به عنوان خروجی
pinMode(LED2, OUTPUT); // تنظیم پین LED ۲ به عنوان خروجی
pinMode(LED3, OUTPUT); // تنظیم پین LED 3 به عنوان خروجی
pinMode(LED4, OUTPUT); // تنظیم پین LED 4 به عنوان خروجی
digitalWrite(LED1, HIGH);
digitalWrite(LED2, HIGH);
digitalWrite(LED3, HIGH);
digitalWrite(LED4, HIGH);
```

تابع `WiFi.begin()` فراخوانی شد و به ورودی آن `SSID` و `Password` مودم مورد نظر به عنوان ورودی داده شد. سپس هنگام شروع اتصال عبارت `Connecting to wifi` در `serial monitor` نوشته می‌شود. سپس یک لوپ وایل نوشته شد. تا زمانی که وضعیت اتصال وای فای به غیر متصل است در این لوپ باقی می ماند و هر ۱۰۰ میلی ثانیه یک نقطه چاپ می کند. وقتی موفق به اتصال شد از این لوپ عبور می‌کند و عبارت `Connected` و `IP` گرفته شده را در `serial monitor` چاپ می‌کند.

می‌توان در تنظیمات مودم تعیین کرد که به این `MAC Address` یک `IP` ثابت اختصاص دهد. با این کار همواره می‌دانیم دستگاه چه `IP` دریافت کرده و همواره موقع باز کردن صفحه وب می‌توان از همین `IP` استفاده کرد.

```
WiFi.begin(WIFI_SSID, WIFI_PASSWORD, WIFI_CHANNEL); // شروع اتصال به شبکه WiFi
Serial.print("Connecting to WiFi ");
Serial.print(WIFI_SSID);
// انتظار برای برقراری اتصال
while (WiFi.status() != WL_CONNECTED) {
  delay(100);
  Serial.print(".");
}
Serial.println(" Connected!");
Serial.print("IP address: ");
Serial.println(WiFi.localIP()); // چاپ آدرس IP محلی
```

تابع `server.on` فراخوانی شد و به ورودی آن `sendHTML` داده شد. پیش تر به تعریف `sendHTML` پرداخته شد.

```
server.on("/", sendHtml); // تنظیم تابع برای ارسال صفحه HTML به روت
server.on(UriBraces("/toggle/{}"), []() {
```

یک `string` به نام `led` تعریف شد.

```
String led = server.pathArg(0);
```

عدد دکمه‌ی فشرده شده در صفحه وب در این string ذخیره می شود.

عدد led که حالت آن برعکس شد در serial monitor چاپ می‌شود.

```
  Serial.print("Toggle LED #");
  Serial.println(led);
  // تغییر وضعیت LED مورد نظر و ارسال مجدد صفحه HTML
```

این string، به نام led با تابع .toInt() به عدد تبدیل شد و به عنوان شرط به switch case داده شد.

```
led.toInt()
```

در switch case در صورت یک بودن led ابتدا مقدار led1state برعکس می شود و در خودش ذخیره می شود. مقدار led1state روی LED1 که پایه متصل به رله ۱ است نوشته می شود. سپس در صورت صفر بودن led1state روی LCD عبارت Relay 1، ON BY WIFI روی LCD چاپ می‌شود و به همین منوال بقیه حالت بندی‌های مطلوب برای همه‌ی کلیدها تعریف شد.

```
  switch (led.toInt()) {
    case 1:
      led1State = !led1State;
      digitalWrite(LED1, led1State);
       if(!led1State){
         lcd.setCursor(0,0);
         lcd.print("Relay 1         ");
         lcd.setCursor(0,1);
         lcd.print("ON BY WIFI      ");
       }
       else {
         lcd.setCursor(0,0);
         lcd.print("Relay 1         ");
         lcd.setCursor(0,1);
         lcd.print("OFF BY WIFI     ");
       }
      break;
    case 2:
      led2State = !led2State;
      digitalWrite(LED2, led2State);
      if(!led2State){
         lcd.setCursor(0,0);
         lcd.print("Relay 2         ");
         lcd.setCursor(0,1);
         lcd.print("ON BY WIFI      ");
       }
```

```
    else {
      lcd.setCursor(0,0);
      lcd.print("Relay 2         ");
      lcd.setCursor(0,1);
      lcd.print("OFF BY WIFI     ");
    }
  break;
case 3:
  led3State = !led3State;
  digitalWrite(LED3, led3State);
  if(!led3State){
     lcd.setCursor(0,0);
     lcd.print("Relay 3         ");
     lcd.setCursor(0,1);
     lcd.print("ON BY WIFI      ");
   }
   else {
     lcd.setCursor(0,0);
     lcd.print("Relay 3         ");
     lcd.setCursor(0,1);
     lcd.print("OFF BY WIFI     ");
   }
  break;
case 4:
  led4State = !led4State;
  digitalWrite(LED4, led4State);
  if(!led4State){
     lcd.setCursor(0,0);
     lcd.print("Relay 4         ");
     lcd.setCursor(0,1);
     lcd.print("ON BY WIFI      ");
   }
   else {
     lcd.setCursor(0,0);
     lcd.print("Relay 4         ");
     lcd.setCursor(0,1);
     lcd.print("OFF BY WIFI     ");
   }
  break;
```

۳.۱.۲ در تابع void loop():

در تابع void loop()، تابع زیر فراخوانی شد (این تابع در هدرهایی که ابتدای برنامه اضافه شد تعریف شده است). این تابع جهات پردازش درخواست وایفای است.

```
server.handleClient(); // پردازش درخواست‌های جدید مشتریان
```

۲.۲ بخش بلوتوث:

۱.۲.۲ ابتدای برنامه، خارج از void setup() و void loop():

متغیر های زیر مربوط به GPIOهای متصل به رله‌ها تعریف شدند. این متغیرها با متغیرهای LED1 الی LED4 مقدار یکسانی دارند. متغیرهای زیر در شروط مربوط به بلوتوث به کار رفت تا بلوتوث و وای فای روی یکدیگر اثر نگذارند.

```
const int relayPin1 =  2;
const int relayPin2 =  26;
const int relayPin3 =  27;
const int relayPin4 =  13;
```

هدرفایل زیر در ابتدای برنامه اضافه شد.

```
#include "BluetoothSerial.h"
```

تعریفات زیر در ابتدای برنامه انجام شد.

```
#if !defined(CONFIG_BT_ENABLED) || !defined(CONFIG_BLUEDROID_ENABLED)
#error Bluetooth is not enabled! Please run `make menuconfig` to and enable it
#endif
#if !defined(CONFIG_BT_SPP_ENABLED)
#error Serial Bluetooth not available or not enabled. It is only available for the
ESP32 chip.
#endif
```

یک string تعریف شد که بعدا برای ذخیره پیام فرستاده شده با بلوتوث به کار خواهد رفت.

```
String message = "";
```

۲.۲.۲ در تابع void setup():

برای تعیین نام دستگاه از کد زیر استفاده می شود (هر نام دلخواهی می تواند داده شود و بعدا با این نام توسط دستگاه‌های دیگر که دارای بلوتوث هستند دیده خواهد شد).

```
SerialBT.begin("ESP32Mandana"); //Bluetooth device name
```

۳.۲.۲ در تابع void loop():

در تابع void loop() تابع bluestart() فراخوانی شد. خارج از تابع loop تابع bluestart تعریف شد کدهای مناسب درون آن اضافه شد.

```
void loop(void) {
  server.handleClient(); // پردازش درخواست‌های جدید مشتریان
  bluestart();
}
void bluestart() {
  if (SerialBT.available()){
    char incomingChar = SerialBT.read();
    if (incomingChar != '\n'){
      message += String(incomingChar);
    }
    else{
      message = "";
    }
    Serial.write(incomingChar);
  }
  // Check received message and control output accordingly
  if (message == "on1"){
    digitalWrite(relayPin1, LOW);
    lcd.setCursor(0,0);
    lcd.print("Relay 1         ");
    lcd.setCursor(0,1);
    lcd.print("ON BY Bluetooth ");
  }
  else if (message == "off1"){
    digitalWrite(relayPin1, HIGH);
    lcd.setCursor(0,0);
    lcd.print("Relay 1         ");
    lcd.setCursor(0,1);
    lcd.print("OFF BY Bluetooth");
  }
if (message == "on2"){
    digitalWrite(relayPin2, LOW);
    lcd.setCursor(0,0);
```

```
    lcd.print("Relay 2         ");
    lcd.setCursor(0,1);
    lcd.print("ON BY Bluetooth ");
  }
  else if (message == "off2"){
    digitalWrite(relayPin2, HIGH);
    lcd.setCursor(0,0);
    lcd.print("Relay 2         ");
    lcd.setCursor(0,1);
    lcd.print("OFF BY Bluetooth");
  }
if (message == "on3"){
    digitalWrite(relayPin3, LOW);
    lcd.setCursor(0,0);
    lcd.print("Relay 3         ");
    lcd.setCursor(0,1);
    lcd.print("ON BY Bluetooth ");
  }
  else if (message == "off3"){
    digitalWrite(relayPin3, HIGH);
    lcd.setCursor(0,0);
    lcd.print("Relay 3         ");
    lcd.setCursor(0,1);
    lcd.print("OFF BY Bluetooth");
  }
if (message == "on4"){
    digitalWrite(relayPin4, LOW);
    lcd.setCursor(0,0);
    lcd.print("Relay 4         ");
    lcd.setCursor(0,1);
    lcd.print("ON BY Bluetooth ");
  }
  else if (message == "off4"){
    digitalWrite(relayPin4, HIGH);
    lcd.setCursor(0,0);
    lcd.print("Relay 4         ");
    lcd.setCursor(0,1);
    lcd.print("OFF BY Bluetooth");
  }
if (message == "on5"){
    lcd.setCursor(0,0);
    lcd.print("Servo Motor     ");
    lcd.setCursor(0,1);
```

```
    lcd.print("180 Degree      ");
   servo1.attach(servo1Pin, 500, 960);
for (pos = 180; pos >= 0; pos -= 1) { // sweep from 180 degrees to 0 degrees
    servo1.write(pos);
    delay(1);
  }
  delay(20);
servo1.detach();
   }
  else if (message == "off5"){
  lcd.setCursor(0,0);
  lcd.print("Servo Motor     ");
  lcd.setCursor(0,1);
  lcd.print("OFF             ");
  ledcFade(LED_PIN, 0,0,LEDC_FADE_TIME);
     }
   if (message == "on6"){
  lcd.setCursor(0,0);
  lcd.print("Servo Motor     ");
  lcd.setCursor(0,1);
  lcd.print("Roating         ");
  servo1.attach(servo1Pin, 1000, 1000);
  for (pos = 0; pos <= 180; pos += 1) { // sweep from 0 degrees to 180 degrees
    // in steps of 1 degree
    servo1.write(pos);
    delay(1);             // waits 20ms for the servo to reach the position
  }
     delay(20);
   }
if (message == "on7"){
  lcd.setCursor(0,0);
  lcd.print("Servo Motor     ");
  lcd.setCursor(0,1);
  lcd.print("45 Degree       ");
   servo1.attach(servo1Pin, 550, 2350);
for (pos = 5; pos >= 0; pos -= 1) { // sweep from 180 degrees to 0 degrees
    servo1.write(pos);
    delay(1);
  }
  delay(20);
servo1.detach();
   }
}
```

اگر پیامی با بلوتوث فرستاده شد یک متغیر کاراکتری تعریف شد اطلاعات خوانده شده با این متغیر برابر قرار داده شد تا زمانی که کاراکتر \n در یافت نشده (نشانه‌ی پایان پیام) پیام را در متغیر incomingchar ذخیره می کند.

```
if (SerialBT.available()){
  char incomingChar = SerialBT.read();
  if (incomingChar != '\n'){
    message += String(incomingChar);
  }
  else{
    message = "";
  }
  Serial.write(incomingChar);
}
```

طبق پیامی که در string، message ذخیره شد حالت بندی و شرط گذاشته شد:

```
if (message == "on1"){
  digitalWrite(relayPin1, LOW);
  lcd.setCursor(0,0);
  lcd.print("Relay 1         ");
  lcd.setCursor(0,1);
  lcd.print("ON BY Bluetooth ");
}
```

در صورت برابر بودن پیام با on1 رله شماره ۱ روشن می شود. و پیام relay 1، ON BY Bluetooth روی LCD چاپ می‌شود. و به همین منوال حالت بندی‌های دیگر برای روشن و یا خاموش کردن دیگر رله‌ها انجام شد.

۳.۲ توضیح LCD:

هدرهای زیر در ابتدای برنامه اضافه شوند.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
```

تابع زیر برای تنظیمات LCD در ابتدای برنامه اضافه شود. LCD می تواند به صورت 20x4 و یا اندازه‌های رایج دیگر مانند 16x2 نیز انجام شود. ورودی اول آدرس I2C را مشخص می‌کند. به صورت پیش فرض و با دادن عدد 0x27 SCL روی ۲۲ و SDA روی ۲۱ قرار دارد.

```
LiquidCrystal_I2C lcd(0x27,20,4);  // set the LCD address to 0x27 for a 20
chars and 4 line display
```

I2C LCD	ESP32
GND	GND
VCC	VIN
SDA	GPIO 21
SCL	GPIO 22

شکل (۲-۱) پایه‌های SDA و SCL میکروکنترلر

۴.۲ توضیح Servo Motor:

هدر فایل زیر اضافه شد.

```
#include <ESP32Servo.h>
```

تعریفات زیر اضافه شد. شماره GPIO متصل به سروو ۵ است. متغیر مربوط به زاویه اولیه آن صفر تعریف شده است.

```
Servo servo1;
int servo1Pin = 5;
int pos = 0;      // position in degrees
```

فصل سوم:

نتایج نهایی پروژه

فصل چهارم:

پیشنهادات

در این بخش به پیشنهاداتی برای بهبود و پیشرفته تر کردن و یا انجام کارهای مشابه با این پروژه پرداخته شده.

۱-پیشنهاد می‌شود از ماژول SIM800 استفاده شود و این پروژه و یا مشابه آن با ارسال پیامک پیاده سازی شود.

۲- پیشنهاد می شود با افزودن یک ماژول رله به پروژه، وسایلی که از برق ولتاژ بالا و AC استفاده می‌کنند را کنترل کرد.

۳-پیشنهاد می شود از سروو موتور برای باز کردن و یا چرخاندن چیزی استفاده شود. به عنوان مثال می‌توان برای باز کردن در سطل زباله و یا خیلی کاربردهای دیگر از سروو موتور، بهره برد.

۴-پیشنهاد می‌شود با استفاده از دوتا ESP32 و پروتکل ESP-NOW که شرکت Espressif روی ESP32ها تعبیه کرده یک ماشین کنترلی ساخت. از یکی از ESP32ها دستورات به ماشین ارسال می‌شود و دیگر ESP32 دستورات مطلوب را اجرا می‌کند.

۵-پیشنهاد می‌شود با استفاده از سروو موتور دست رباتیک و ... ساخته شود.

برآورد هزینه این پروژه در جدول ۴-۱ آورده شده است.

برآورد هزینه پروژه:

شماره	قطعه	هزینه تقریبی (تومان)
۱	برد ESP32	۲۶۰۰۰۰
۲	کابل Micro_USB	۱۸۰۰۰
۳	ماژول رله ۵ ولت ۴ کاناله	۱۸۹۰۰۰
۴	LCD کاراکتری 20x4 +I2C	۲۲۵۰۰۰
۵	سروو موتور	۷۸۰۰۰
۶	منبع تغدیه	۲۴۰۰۰۰
۷	برد بورد	۱۰۰۰۰۰

۸	سیم جامپر	۴۵۰۰۰
۹	سیم ۱.۵mm	۶۰۰۰۰
۱۰	لامپ	۲۲۰۰۰۰
۱۱	سرپیچ دیواری	۶۰۰۰۰
۱۲	دوشاخه	۲۵۰۰۰
۱۳	جمع	۱۵۲۰۰۰۰

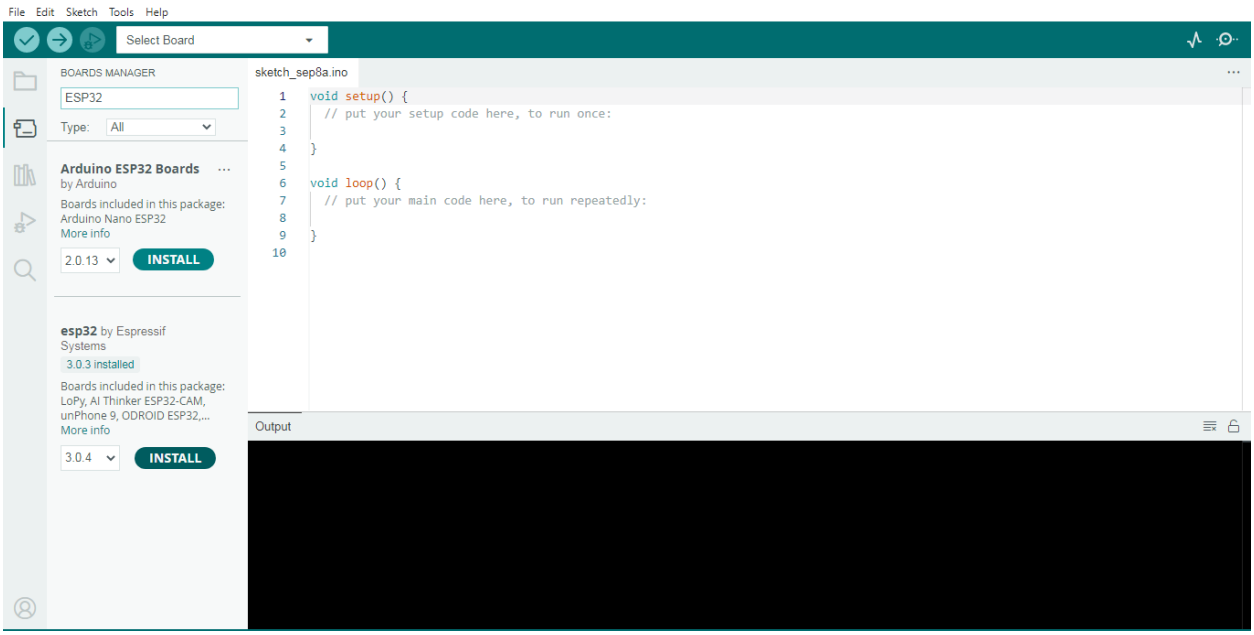
جدول (۴-۱) برآورد هزینه پروژه

فصل پنجم:

پیش نیازهای نرم افزاری

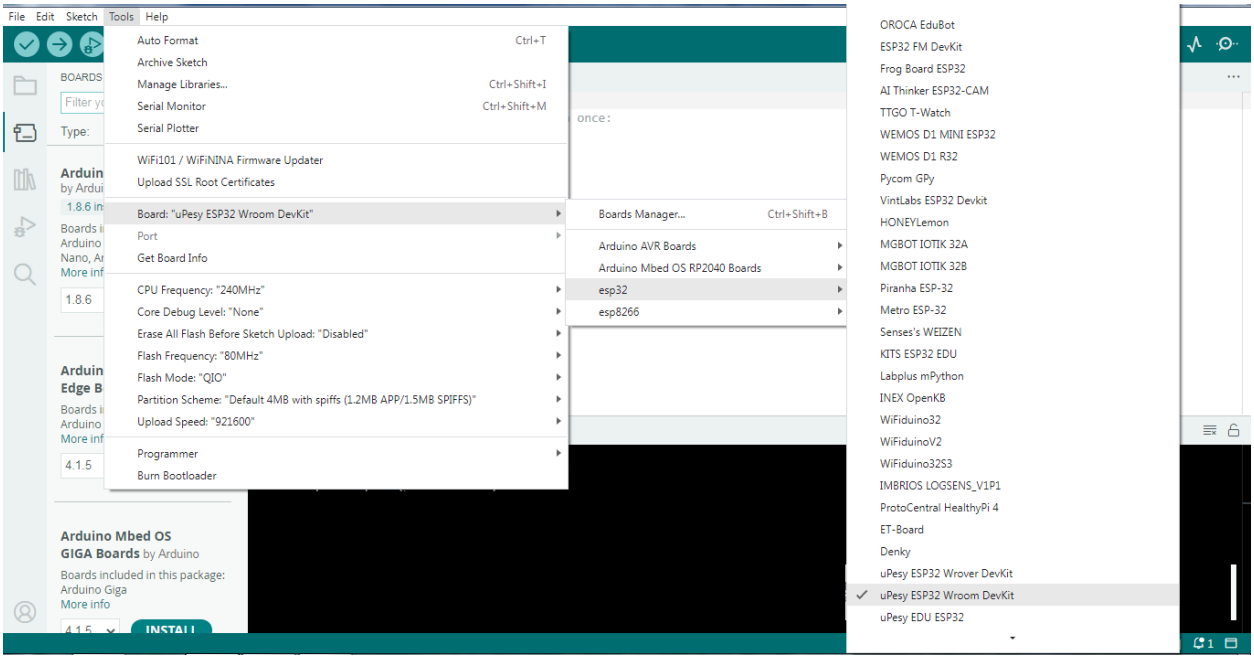
۱.۵ نصب Arduino IDE:

نرم افزار Arduino IDE نصب شود. در قسمت Board Manager، ESP32 جست و جو و دانلود شد.



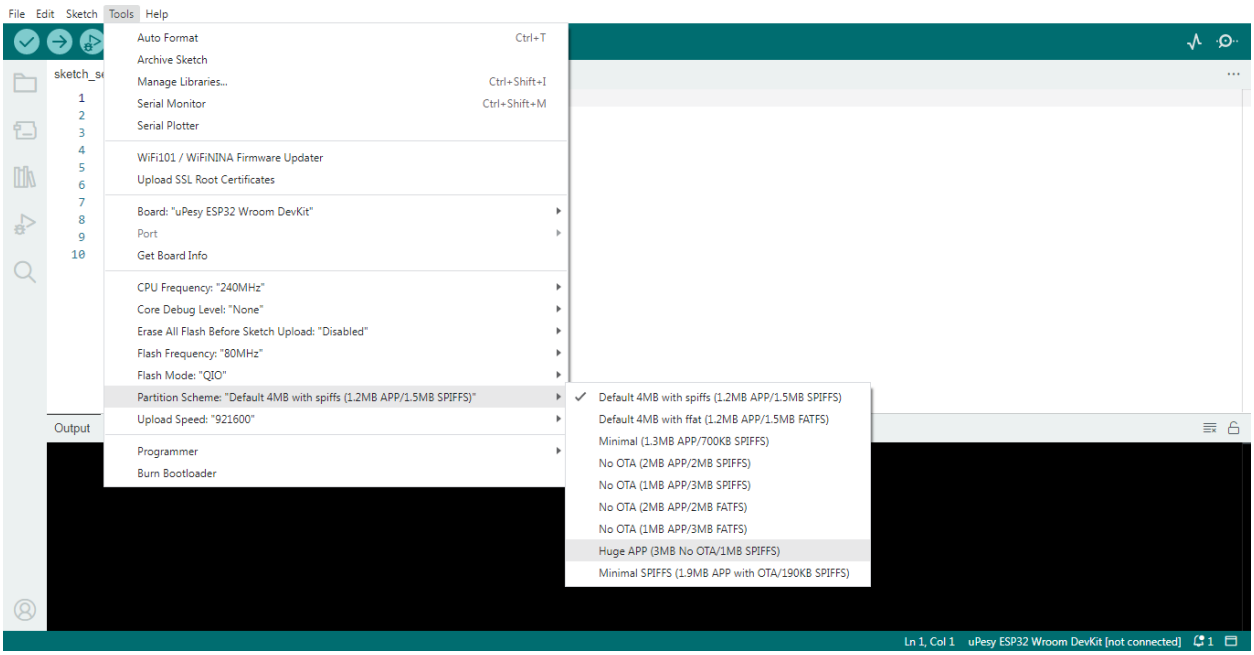
شکل (۵-۱) نصب کتابخانه‌ی برد ESP32

از منو Tools، Board، ESP32، uPesy ESP32 WROOM DevKit، انتخاب شود.



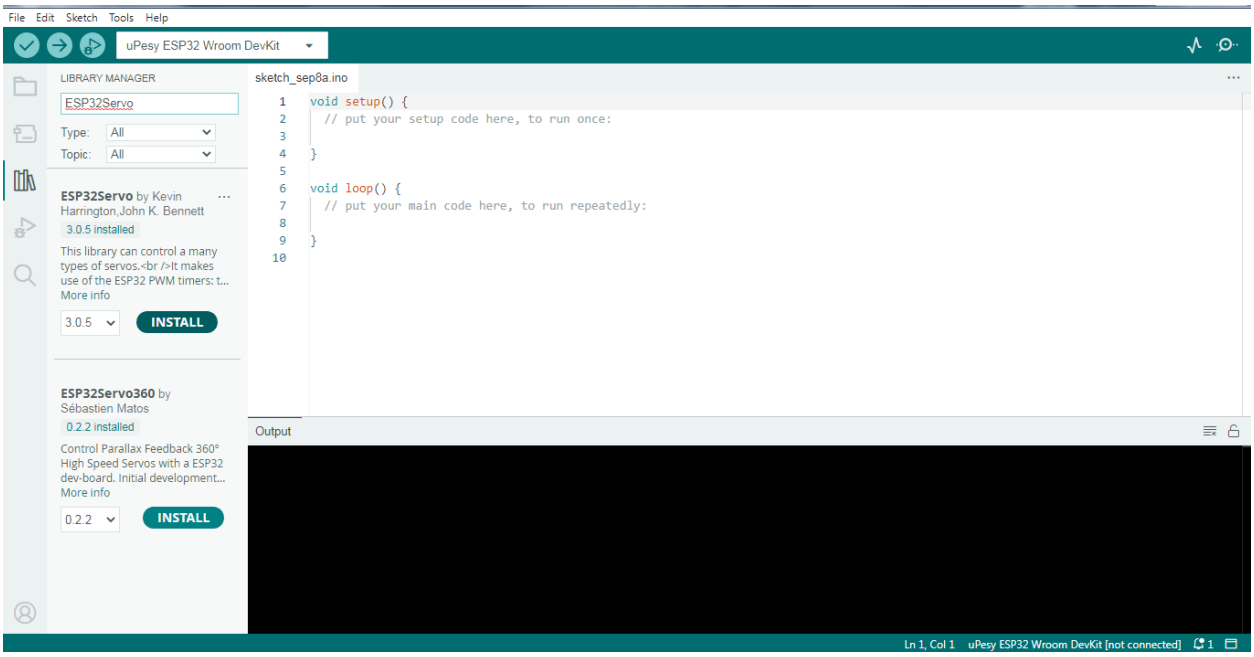
شکل (۵-۲) انتخاب برد uPesy ESP32 WROOM DevKit

در صورتی که برنامه خیلی بزرگ شد، از حافظه داخلی ۳ مگابایت به ذخیره برنامه اختصاص داده شود.



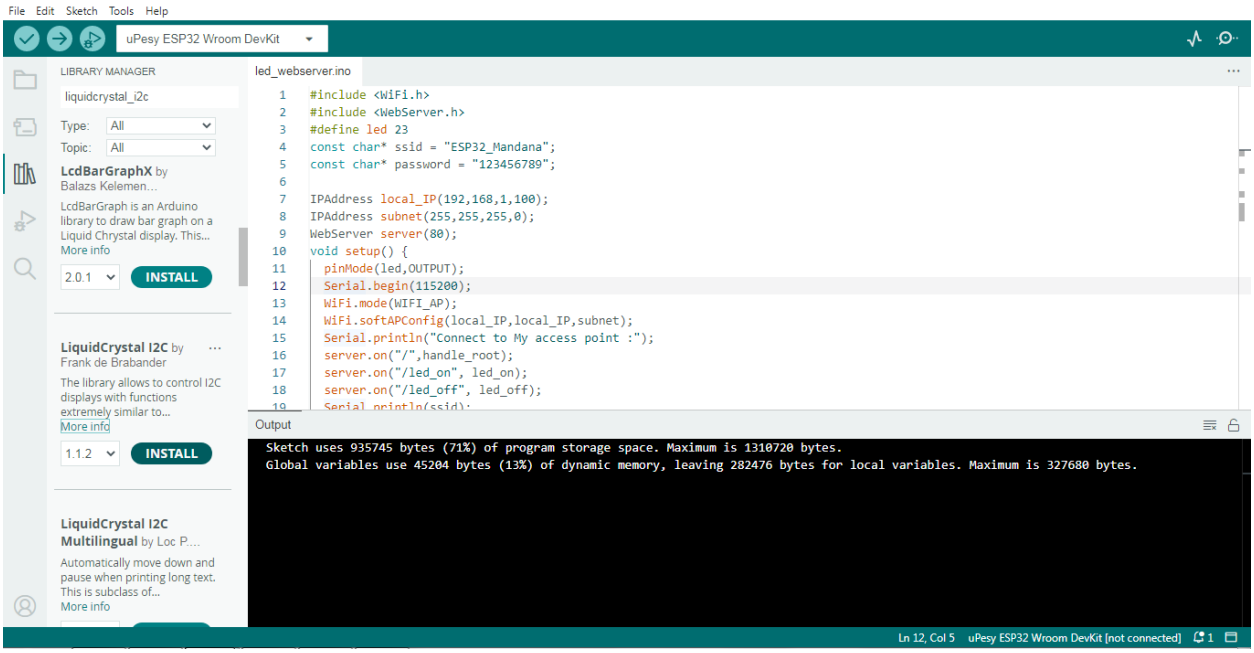
شکل (۵-۳) افزایش اندازه حافظه اختصاص داده شده به کد

کتابخانه‌ی ESP32Servo از قسمت Library Manager دانلود شود.



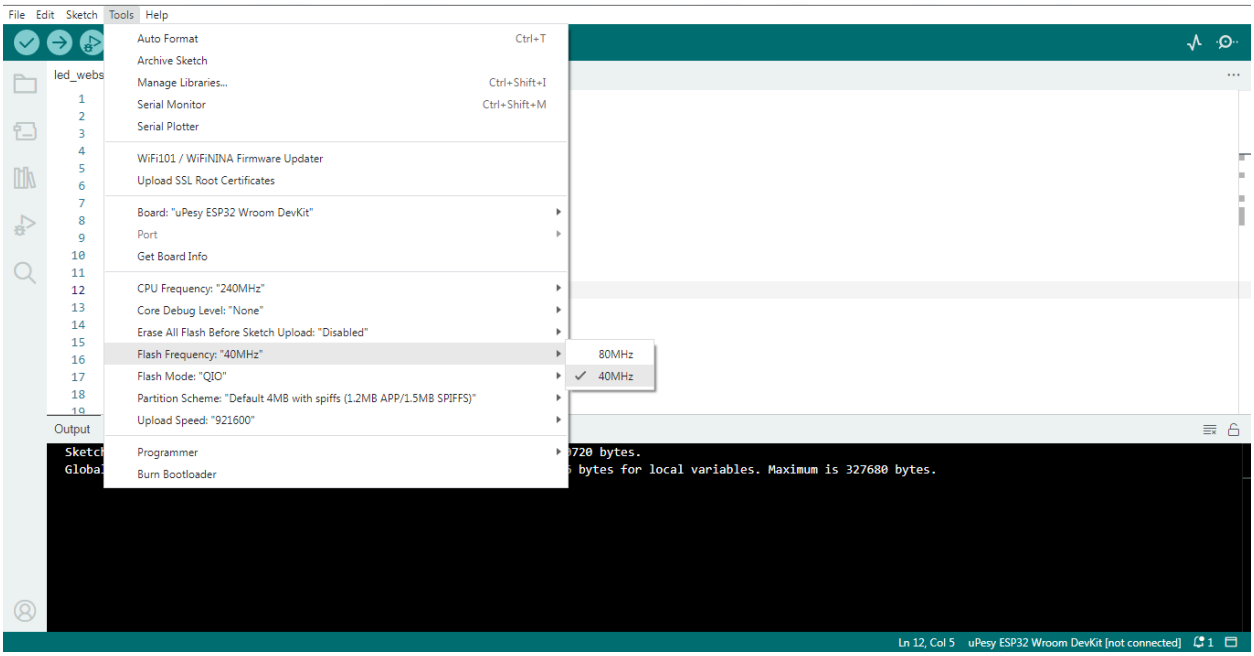
شکل (۵-۴) نصب کتابخانه ESP32Servo.h

کتابخانه LiquidCrystal_I2C از قسمت Library Manager نصب شود.



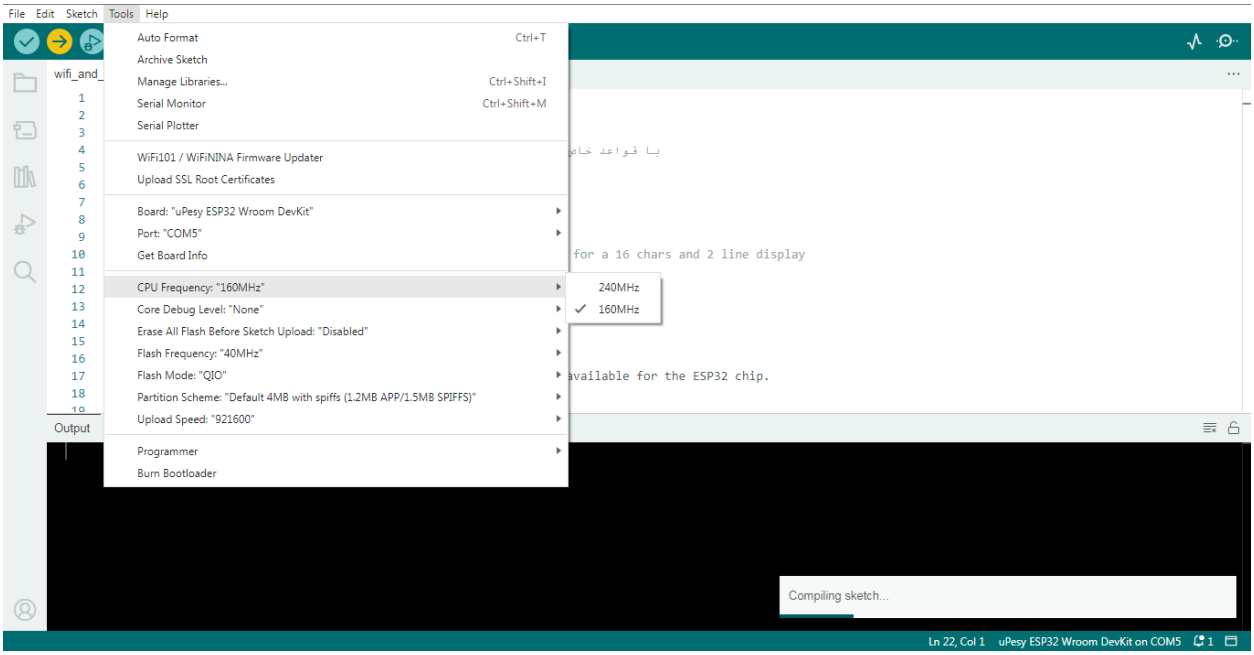
شکل (۵-۵) نصب کتابخانه LiquidCrystal_I2C

Flash Frequency روی ۴۰MHz تنظیم شود.



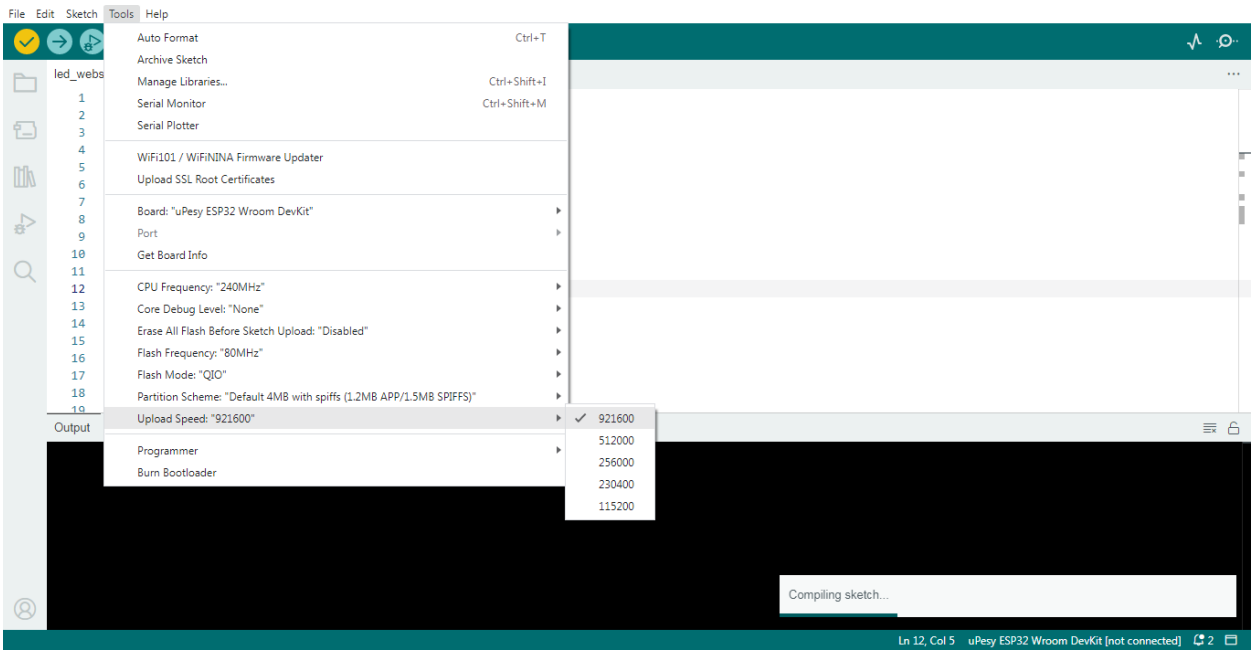
شکل (۵-۶) تغییر فرکانس flash frequency

CPU Frequency روی ۱۶۰MHz تنظیم شود.



شکل (۵-۷) تغییر فرکانس CPU به ۱۶۰MHz

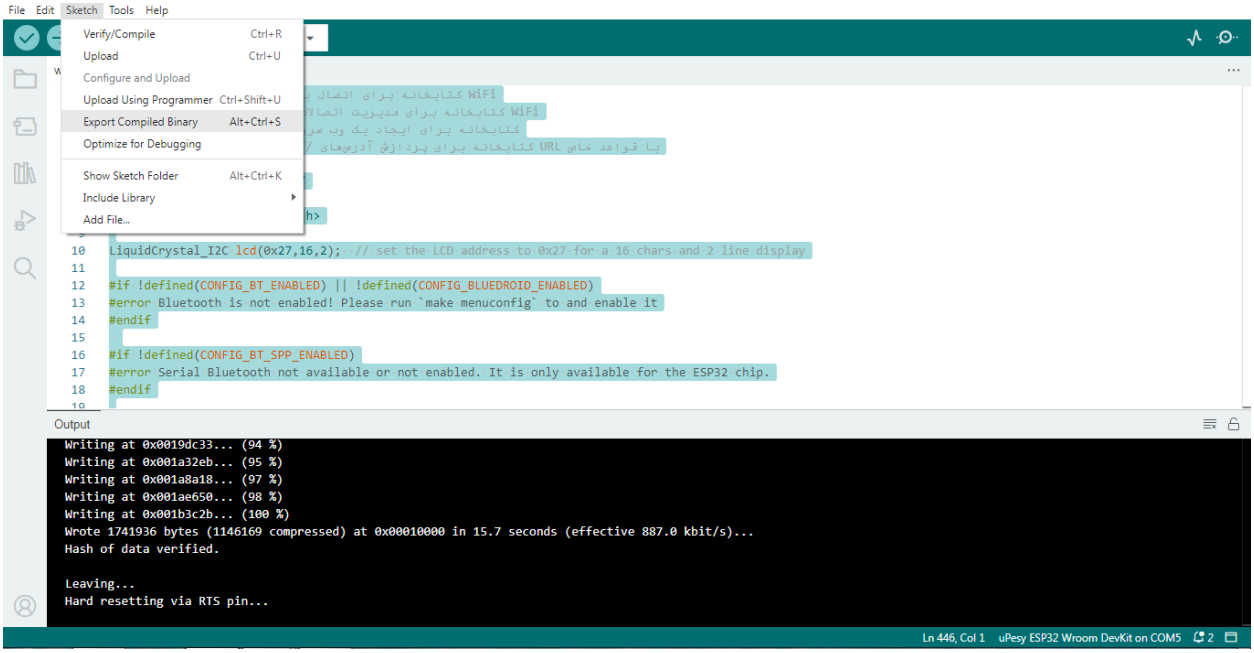
در این قسمت می‌توان بادریت پروگرمینگ را تنظیم کرد.



شکل (۵-۸) تنظیم بادریت پروگرمینگ

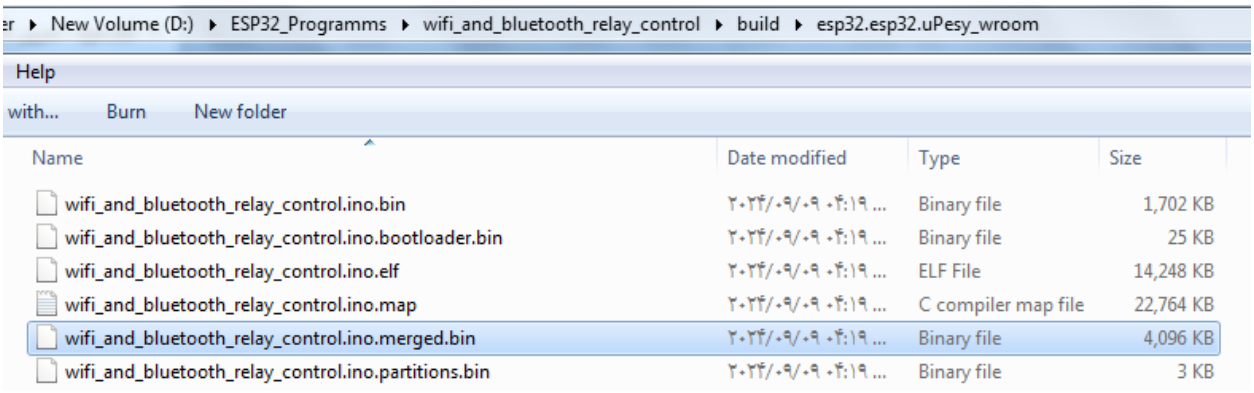
در صورت نیاز می‌توان از قسمت، sketch، Export Compiled Binary، فایل را کامپایل و ذخیره کرد.

می‌توان این فایل را با برنامه های فلشر روی میکروکنترلر پروگرم کرد.



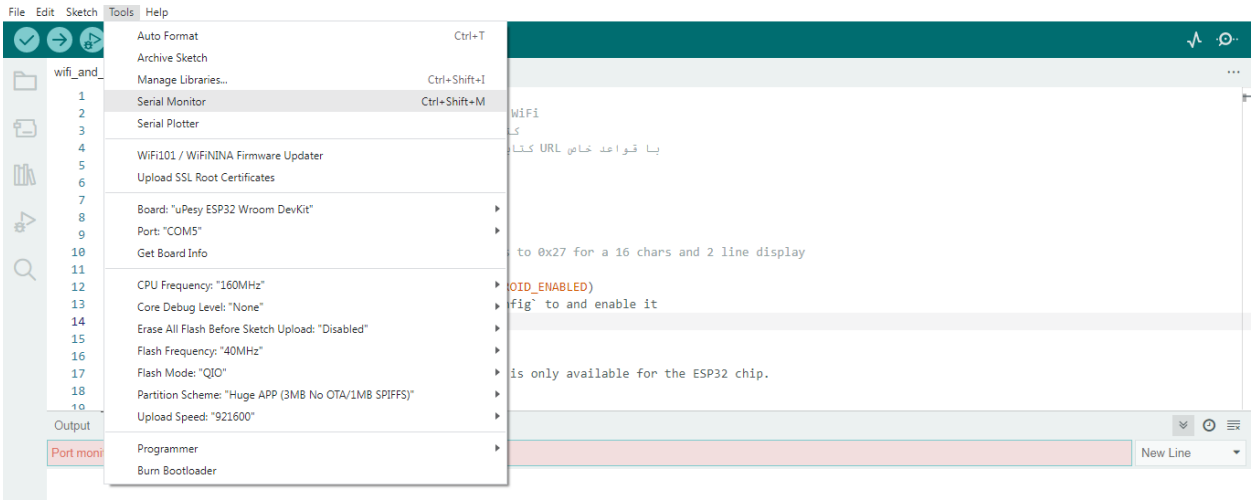
شکل (۵-۹) ایجاد فایل باینری کد

این فایل که اندازه آن برابر ۴MB است (اندازه حافظه میکروکنترلر) می تواند با یک برنامه فلشر، روی میکروکنترلر پروگرم شود.



شکل (۵-۱۰) فایل باینری قابل فلش روی برد

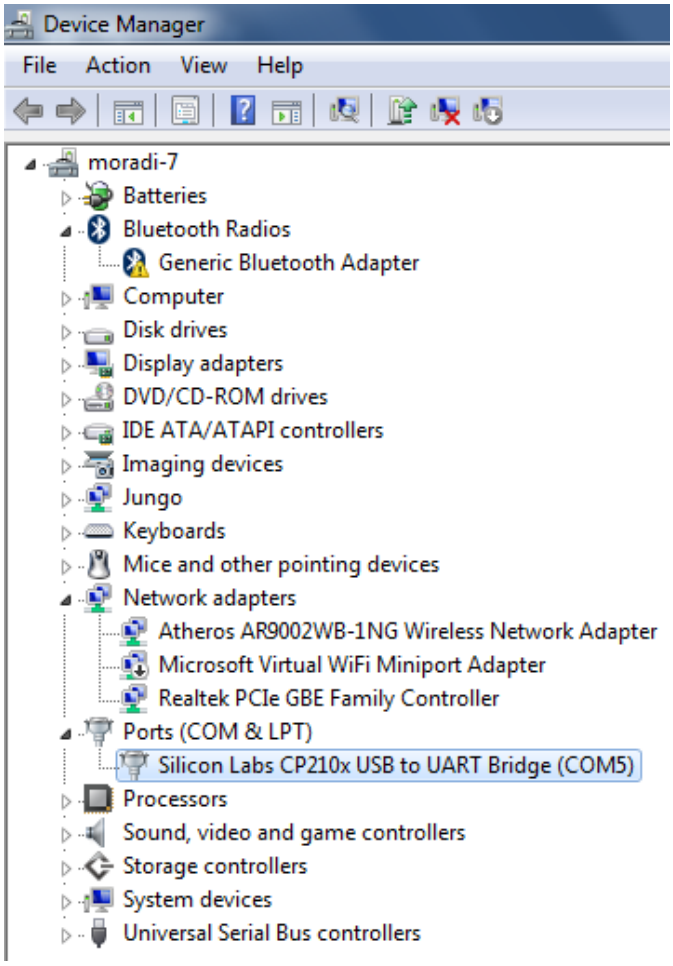
از این بخش می‌توان به ترمینال سریال را باز کرد و داده های ارسال شده از طرف برد را دریافت کرد.



شکل (۵-۱۱) نحوه‌ی باز کردن Serial Monitor در Arduino IDE

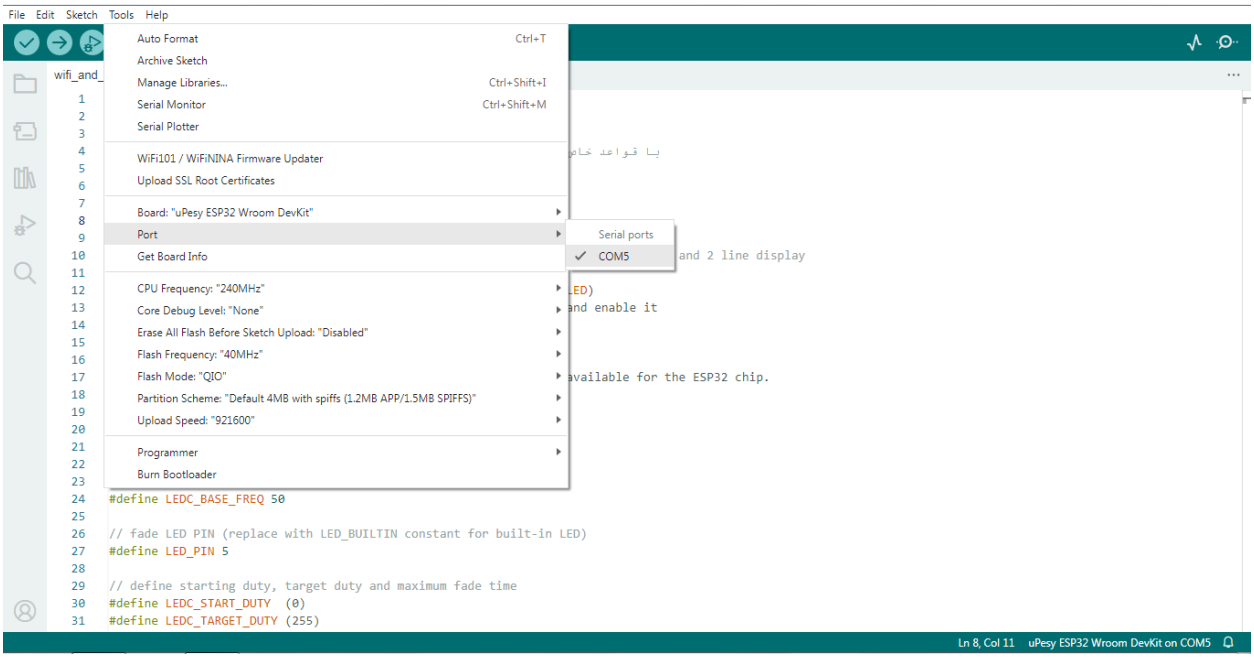
۲.۵ نصب درایور نرم افزار CP2102:

پیش از پروگرم کردن، برای اینکه برد توسط ماکپیوتر قابل شناسایی باشد، باید نرم افزار درایور CP2102 نصب شود. سپس در قسمت Device Manager در Windows شماره COM متصل به مبدل CP2102 روی برد بررسی شود.



شکل (۵-۱۲) مشاهده شماره COM در Device Manager

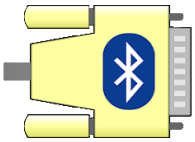
شماره این COM در قسمت Tools، PORT، انتخاب شود.



شکل (۵-۱۳) انتخاب شماره COM در Arduino IDE

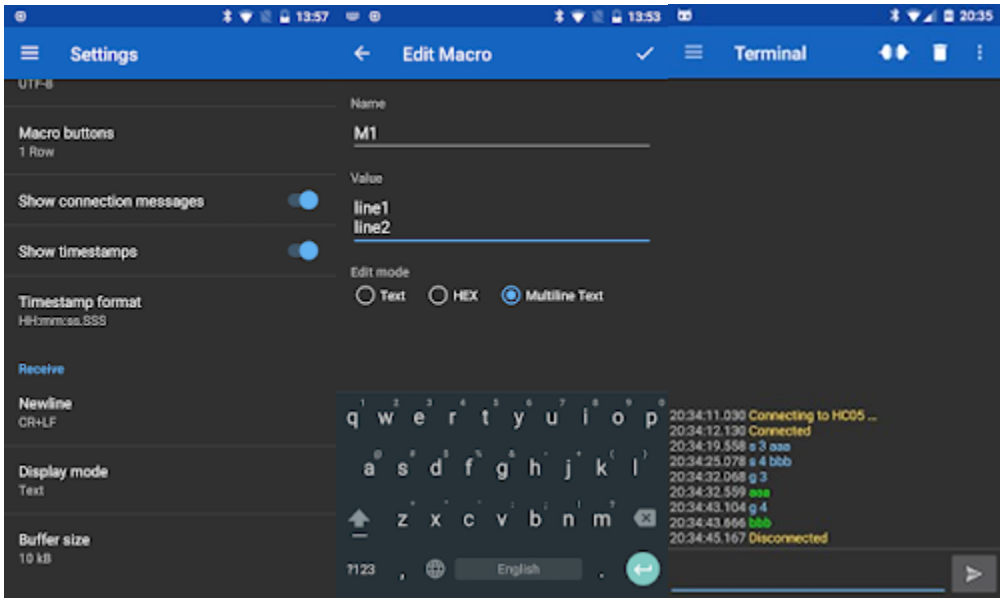
۳.۵ نصب نرم افزار موبایل برای ارتباط با بلوتوث:

برای ارتباط با برد از طریق بلوتوث موبایل برنامه Serial Bluetooth Terminal استفاده شد.



شکل (۵-۱۴) آیکون نرم افزار serial Bluetooth terminal

برای تعریف دکمه‌ها به صورت زیر عمل شود. از منو settings، در قسمت Macro Buttons، یک ردیف به دو ردیف تغییر تغییر یابد. در قسمت ترمینال روی کلیدها چند ثانیه مکث شود تا صفحه تنظیمات یک کلید باز شود. در این صفحه نام و مقدار (value) دکمه مشخص شود. value مقداری است که به صورت سریال با بلوتوث برای ESP32 فرستاده می‌شود.



شکل (۵-۱۵) نرم افزار Serial Blutooth Terminal

مراجع:

Espressif online flasher:

https://espressif.github.io/esptool-js/

Espressif Flash Download Tools:

https://www.espressif.com/en/support/download/other-tools

Espressif ESP32 Firmware Download:

https://docs.espressif.com/projects/esp-at/en/latest/esp32/AT_Binary_Lists/esp_at_binaries.html

Datasheets:

https://docs.espressif.com/projects/esp-idf/en/stable/esp32/hw-reference/esp32/get-started-devkitc.html

node mcu flasher:

https://nodemcu.readthedocs.io/en/dev-esp32/flash/

کد وای فای:

https://xeicub.ir/%D9%82%D8%AF%D9%85-%D8%B3%D9%88%D9%85-%D8%B3%D8%B1%D9%88%D8%B1-http-esp32/

کد بلوتوث کلاسیک:

https://roboeq.ir/blog/%D8%B1%D8%A7%D9%87-%D8%A7%D9%86%D8%AF%D8%A7%D8%B2%DB%8C-%D8%A8%D9%84%D9%88%D8%AA%D9%88%D8%AB-%DA%A9%D9%84%D8%A7%D8%B3%DB%8C%DA%A9-esp32/#:~:text=%D8%A2%D9%85%D9%88%D8%B2%D8%B4%20%D8%A7%D8%AA%D8%B5%D8%A7%D9%84%20ESP32%20%D8%A8%D9%87%20%D8%AA%D9%84%D9%81%D9%86%20%D9%87%D9%85%D8%B1%D8%A7%D9%87%20%D8%A7%D9%86%D8%AF%D8%B1%D9%88%DB%8C%D8%AF%20%D8%A8%D8%A7%20%D8%A8%D9%84%D9%88%D8%AA%D9%88%D8%AB%20%DA%A9%D9%84%D8%A7%D8%B3%DB%8C%DA%A9&text=%DB%B1%2D%20%D8%A7%D8%A8%D8%AA%D8%AF%D8%A7%20%D8%AA%D8%BA%D8%AF%DB%8C%D9%87%20%D9%85%D8%A7%DA%98%D9%88%D9%84%20ESP32,%D8%A8%D8%B1%D8%A7%DB%8C%20%DA%86%D9%86%D8%AF%20%D8%AB%D8%A7%D9%86%DB%8C%D9%87%20%D9%85%D9%86%D8%AA%D8%B8%D8%B1%20%D8%A8%D9%85%D8%A7%D9%86%DB%8C%D8%AF.

کد PWM:

https://docs.espressif.com/projects/arduino-esp32/en/latest/api/ledc.html

https://docs.espressif.com/projects/arduino-esp32/en/latest/migration_guides/2.x_to_3.0.html

کتابخانه و نحوه ی کار با LCD:

https://randomnerdtutorials.com/esp32-esp8266-i2c-lcd-arduino-ide/

https://github.com/johnrickman/LiquidCrystal_I2C

لینک گوگل پلی ترمینال بلوتوث برای موبایل:

https://play.google.com/store/apps/details?id=de.kai_morich.serial_bluetooth_terminal&hl=en&pli=1

Robot arm:

https://www.youtube.com/watch?v=ZEir102PxJ8

https://www.youtube.com/watch?v=K7cXGwFUkkY